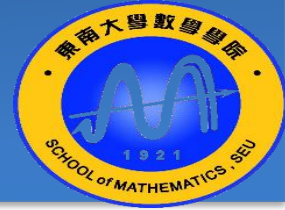




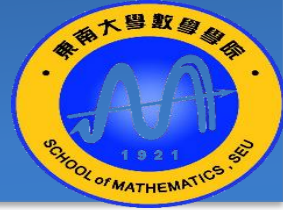
8.3图论/网络优化问题



1. 图论基本概念
2. SRP、MSTP、PERT/CPM 标准模型及其求解算法
3. 用 Matlab、LINGO 软件求解典型图论优化问题



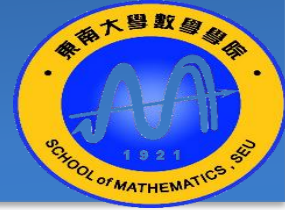
§ 1 图的基本概念



- **图**(Graph) 是由**顶点**集合 $V(G)$ 和**边**的集合 $E(G)$ 组成的数据结构, 通常用 $G = \langle V, E \rangle$ 来表示, 边表示顶点间的关系。
- 顶点个数称为图的**阶**(Order), 通常用 $v(G) = n$ 表示。
- 边的个数称为图的**边数**(Size), 通常用 $\varepsilon(G) = m$ 表示。
- 边 e 如果没有方向, 则称为**无向边**, 用无序对 (u, v) 表示; 如果有方向, 则称为**有向边**或**弧**, 用有序对 (u, v) 表示, u 称为 e 的**起点**, v 称为 e 的**终点**, u 和 v 统称为 e 的**端点**。称 e **关联** u 和 v , u 和 v 是**邻接点**。
- 关联同一个顶点的两条边, 称为**邻边**。端点重合的一条边, 称为**自环**。



§ 2 有向/无向图与度



- 所有边都是有向边的图称**有向图**，所有边都是无向边的图称**无向图**。既有有向边，又有无向边的图称**混合图**。
- 无向图中，两顶点间 (包括自身之间) 若有多条边连接，则称这几条边为**多重边**。
- 在有向图中，两顶点间 (包括自身之间) 若有多条**同起点**和**同终点**的边，则称这几条边为**多重边**。
- 既没有多重边又没有环的图称为**简单图**，含有多重边的图称为**多重图**。
- 无向图中，顶点 v 作为边的端点的次数为 v 的**度**，记作 $d(v)$ 。**环的度为 2**。
- 有向图中，顶点 v 作为边的起点的次数为 v 的**出度**，记为 $d^+(v)$ ，作为边的终点的次数为 v 的**入度**，记为 $d^-(v)$ ， v 的**度**是出度和入度之和。

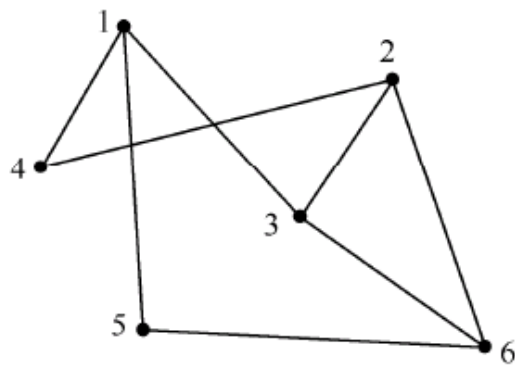
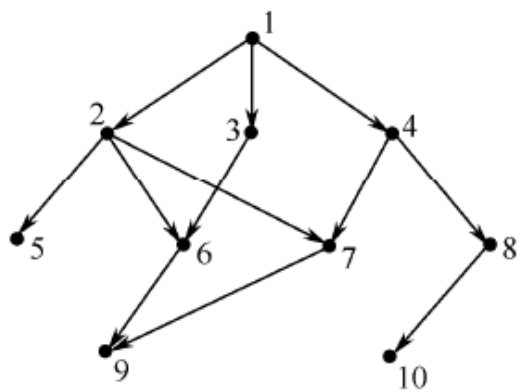
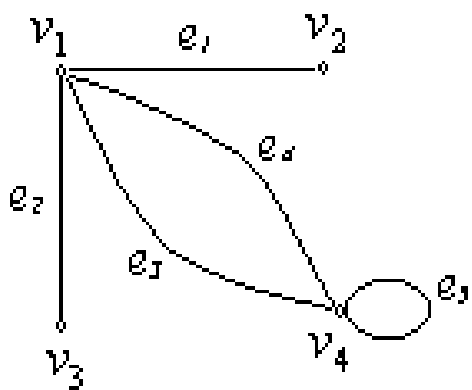
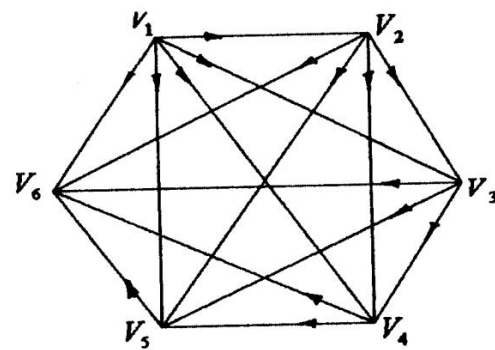


图 8.1 有向图与无向图



$$d(v_4) = 4$$



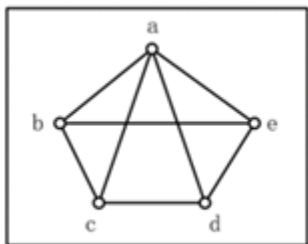
$$d^+(v_4) = 2$$

$$d^-(v_4) = 3$$

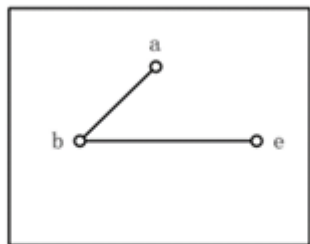
$$d(v_4) = 5$$

设 $G = \langle V, E \rangle$ 和 $G' = \langle V', E' \rangle$ 是两个图。

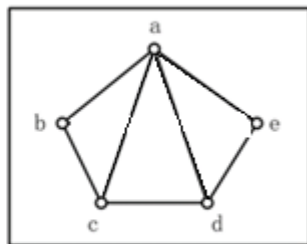
- 如果 $V' \subseteq V$ 且 $E' \subseteq E$, 则称 G' 是 G 的**子图**, 记作 $G' \subseteq G$.
- 如果 $G' \subset G$, 则称 G' 是 G 的**真子图**.
- 如果 $V' = V$ 且 $G' \subseteq G$, 则称 G' 是 G 的**生成子图**.



a) 图 G



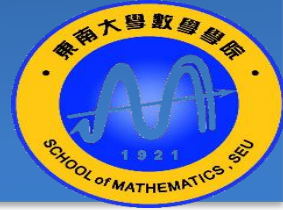
b) 真子图



c) 生成子图



§ 4 连通性

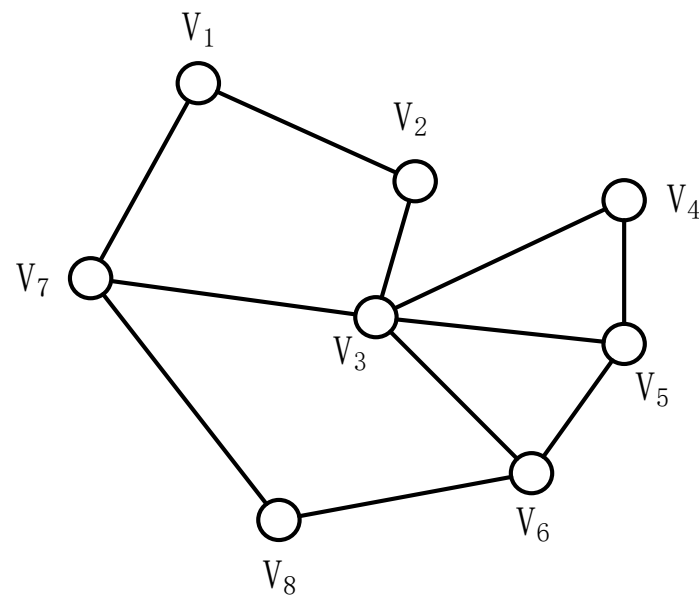


给定无向图（有向图） $G = \langle V, E \rangle$, 令 $v_0, v_1, \dots, v_k \in V$, 边（有向边） $e_1, \dots, e_k \in E$, 其中 $e_i = (v_{i-1}, v_i)$, 交替序列 $v_0 e_1 v_1 e_2 v_2 \dots e_k v_k$ 称为连接起点 v_0 和终点 v_k 的通路 (walk) (或链), 起点终点重合时, 称为回路 (circuit). 边的数目 k 称为链的长度.

- 简单通路 (trail) 或道路/轨迹: 所有边不重复的通路. 有通路必有道路.
- 简单回路: 所有边不重复的回路
- 路径 (path): 所有顶点不重复（从而所有边不重复）的通路
- 圈 (cycle): 除了 $v_0 = v_k$ 外, 所有顶点不重复（从而所有边不重复）的回路.
- 路径 (或圈) 一定是简单通路 (或简单回路), 反之则不一定.

例 1. 在图中 (不引起歧义的前提下省略边), 注意几种概念的区别.

1. $v_1v_2v_3v_4v_3$ 是一条 4-通路
2. $v_1v_2v_3v_4v_5v_6v_3v_7$ 是一条 7-道路
3. $v_1v_2v_3v_4v_5$ 是一条 4-路径
4. $v_1v_2v_3v_4v_5v_6v_3v_7v_1$ 是一个回路.
5. $v_1v_2v_3v_4v_5v_6v_8v_7v_1$ 是一个圈.





§ 4 连通性



设 u, v 为图 $G = \langle V, E \rangle$ 中的两个顶点。

1. 若 G 为无向图, u, v 之间存在通路, 则称顶点 u, v 是**连通**的, 记为 $u \sim v$ 。
2. 如果从 u 到 v 存在一条路径, 则称从 u **可达** v , 记为 $u \rightarrow v$ 。
3. 如果 u, v **相互可达**, 则记为 $u \leftrightarrow v$ 。
4. 对任意顶点 u , 规定 $u \sim u, u \rightarrow u, u \leftrightarrow u$ 。

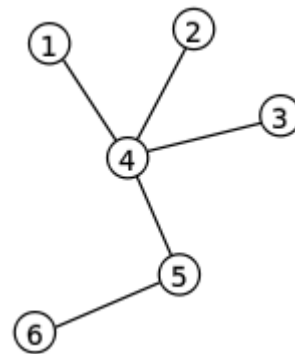
若无向图 $G = \langle V, E \rangle$ 中任意两个顶点都是连通的, 则称 G 是**连通图**, 否则称 G 是**非连通图**。

若有向图 $G = \langle V, E \rangle$ 中任何一对顶点之间都是相互可达的, 则称 G 是**强连通图**。

若有向图 $G = \langle V, E \rangle$ 对应的**底图**(忽略边的方向对应的无向图) 是**连通图**, 则称 G 是**连通图**或**弱连通图**, 否则称 G 是**非连通图**。

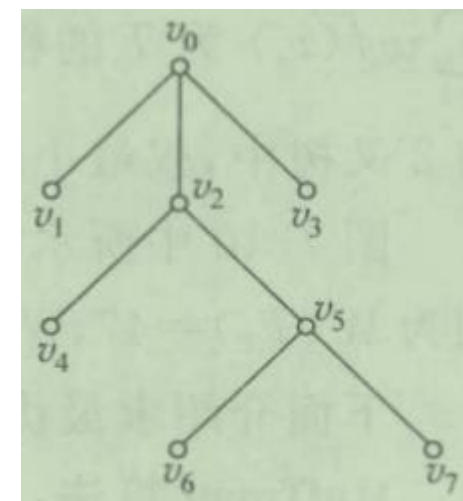
树 (tree)

1. 无向树: 连通的不含圈的无向图.
2. 有向树: 底图是树的有向图.



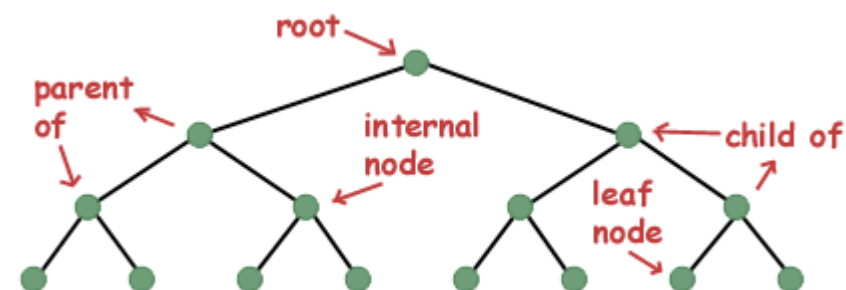
定义: (根树) 一棵非平凡的有向树, 如果恰有一个顶点的入度为 0, 其余顶点的入度均为 1, 则称此有向树为**根树**. 入度为 0 的顶点称为**树根** (root), 入度为 1、出度为 0 的顶点称为**树叶**, 入度为 1、出度大于 0 的顶点称为**内点**, 内点和树根统称为**分支点**. 从树根到顶点 v 的通路长度称为 v 的**层数**, 记为 $l(v)$. 称层数相同的顶点在同一层上. 顶点的最大层数称为 T 的**树高**, 记为 $h(T)$.

在画根树时, 通常把树根放在最上方, 边的方向都向下, 可省略箭头, 如图所示. v_0 为树根, v_1, v_3, v_4, v_6, v_7 为树叶, v_2, v_5 为内点, v_0, v_2, v_5 为分支点. v_0 在第 0 层上, v_1, v_2, v_3 在第 1 层上, v_4, v_5 在第 2 层上, v_6, v_7 在第 3 层上, 树高为 3, 在树叶 v_6, v_7 达到这个高度.



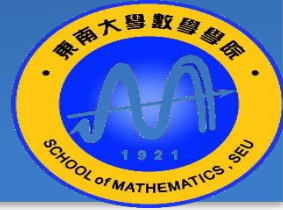
一棵根树可以看成一棵**家族树**:

- (1) 若顶点 a 指向顶点 b , 则称 a 为 b 的父亲, b 为 a 的儿子;
- (2) 若 a, b 有同一个父亲, 则称 a, b 为兄弟;
- (3) 若 $a \neq b$, 而 $a \rightarrow b$, 则称 a 为 b 的祖先, b 为 a 的后代. root 为家族的始祖.





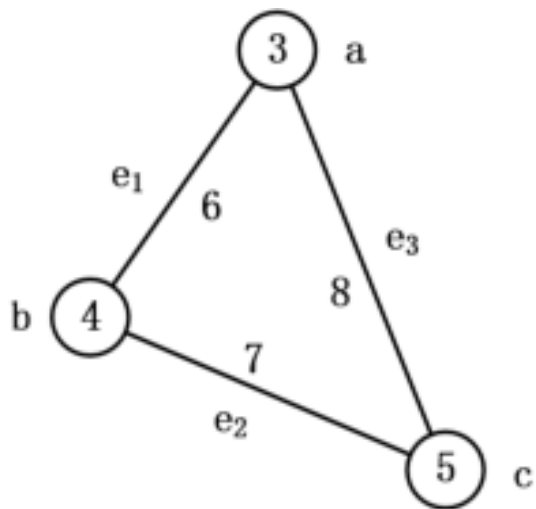
§ 5 赋权图



定义：给每条边都赋予权的图 $G = \langle V, E \rangle$ 称为**赋权图**(weighted graph)。 $w(e)$ 表示边 e 的权。 $w(P)$ 表示路径 P 的权, 即路径 P 的各边权值之和。 $w(T)$ 表示树 T 的权, 即树 T 各边权值和.

最短路 SRP(Shortest Route Problem)

在赋权图 G 中, 从顶点 u 到顶点 v 的具有最小权的路径, 称为 u 到 v 的最短路.



最小生成树 MSTP(Minimum Spanning Tree Problem)

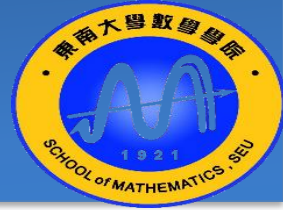
在赋权图 G 中, 求权最小的生成树.



§ 6 矩阵表示



- 邻接矩阵（顶点间的关系）
- 关联矩阵（顶点与边的关系）
- 连通性与矩阵关系（判断顶点间是否有路径）



简单图无环无重边

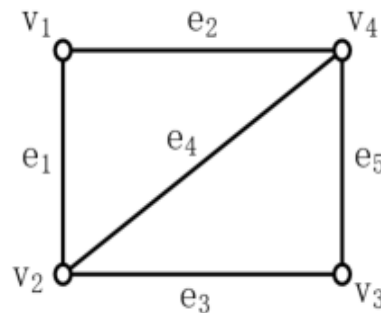
n 阶 **无权简单图** G 的邻接矩阵 $A(G)$ 是一个 n 阶方阵 $A(G) = (a_{ij})_{n \times n}$,
其中

$$a_{ij} = \begin{cases} 1, & (v_i, v_j) \in E \\ 0, & (v_i, v_j) \notin E \end{cases}$$

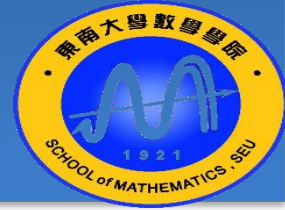
简单无权图邻接矩阵的**性质**:

- (1) **对角线元素全为零。**
- (2) 无向图中, 顶点 i 的度 = 第 i 行行和 = 第 i 列列和
- (3) 有向图中, 顶点 i 的出度 = 第 i 行行和, 顶点 i 的入度 = 第 i 列列和。
- (4) **无向图邻接矩阵对称, 有向图邻接矩阵不一定对称。**

例 1. 求下图 G 的邻接矩阵 A .



解: $A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix},$



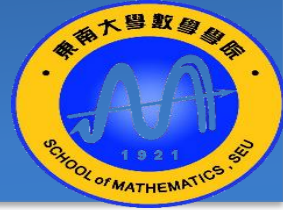
定理: 设 $G = \langle V, E \rangle$ 是简单图, $V = \{v_1, v_2, \dots, v_n\}$, G 的邻接矩阵 $A = (a_{ij})_{n \times n}$, 则

1. 如果 G 为无向图, 则

$$d(v_i) = \sum_{j=1}^n a_{ij} = \sum_{j=1}^n a_{ji}$$

2. 如果 G 为有向图, 则

$$d^+(v_i) = \sum_{j=1}^n a_{ij}, \quad d^-(v_i) = \sum_{j=1}^n a_{ji}$$



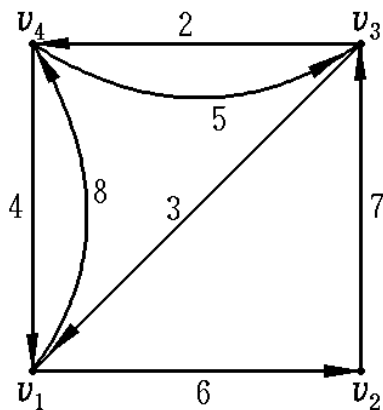
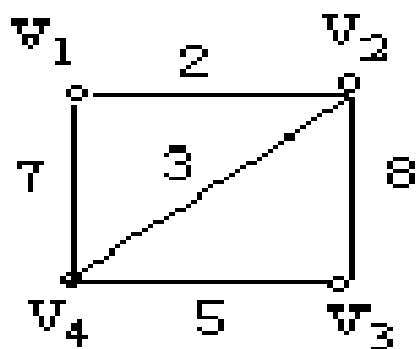
赋权简单图的邻接矩阵

n 阶赋权简单图 G 的邻接矩阵 $A(G)$ 是一个 n 阶方阵 $A(G) = (a_{ij})_{n \times n}$, 其中

$$a_{ij} = \begin{cases} w_{ij}, & (v_i, v_j) \in E \\ \text{特殊值}, & (v_i, v_j) \notin E \end{cases}$$

这里特殊值根据实际问题背景来定, 一般取 $+\infty, -\infty, 0$.

例 3. 求下面的赋权图的邻接矩阵.



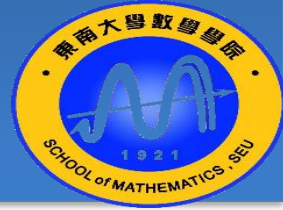
解: 其中一种邻接矩阵

$$A_1 = \begin{pmatrix} \infty & 2 & \infty & 7 \\ 2 & \infty & 8 & 3 \\ \infty & 8 & \infty & 5 \\ 7 & 3 & 5 & \infty \end{pmatrix},$$

$$A_2 = \begin{pmatrix} \infty & 6 & \infty & 8 \\ \infty & \infty & 7 & \infty \\ 3 & \infty & \infty & 2 \\ 4 & \infty & 5 & \infty \end{pmatrix}$$



关联矩阵



如果 n 阶无环图 G 有 m 条边, 则它的关联矩阵 $M(G)$ 是一个 $n \times m$ 阶矩阵 $M(G) = (m_{ij})_{n \times m}$, 其中

1. G 是有向图时,
$$m_{ij} = \begin{cases} 1, & v_i \text{ 是 } e_j \text{ 的起点} \\ -1, & v_i \text{ 是 } e_j \text{ 的终点} \\ 0, & \text{其它} \end{cases}$$

2. G 是无向图时,
$$m_{ij} = \begin{cases} 1, & v_i \text{ 与 } e_j \text{ 关联} \\ 0, & v_i \text{ 与 } e_j \text{ 不关联} \end{cases}$$

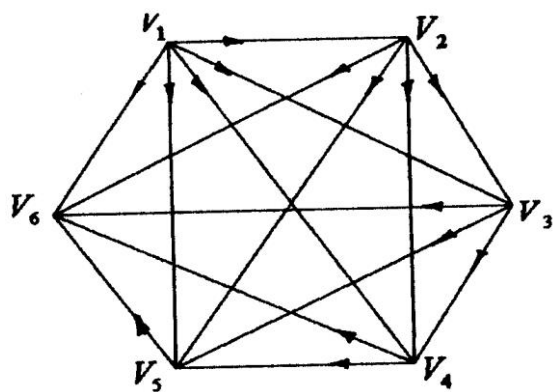
无向图关联矩阵有如下性质:

1. 第 i 行 ($1 \leq i \leq n$) 中, 1 的个数是 v_i 的度;
2. 每列都恰有两个 1;

有向图关联矩阵有如下性质:

1. 第 i 行 ($1 \leq i \leq n$) 中, 1 的个数是 v_i 的出度, -1 的个数是 v_i 的入度;
2. 每列都恰有一个 1 和一个 -1 ;

例 4. 求下面图的邻接矩阵 A 和关联矩阵 M .



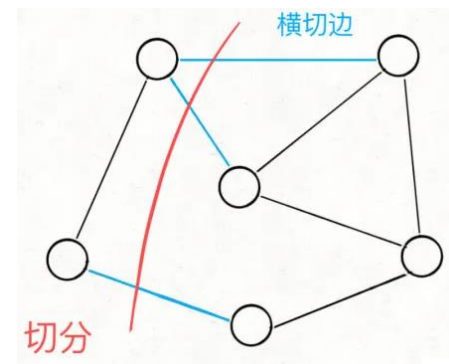
解: $A = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix},$

$$M = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & 0 & -1 & -1 \end{pmatrix}$$

定义（图的割边）：对于图 $G = \langle V, E \rangle$ ，对任意非空顶点集 $P \subseteq V$ ，那么割集为：

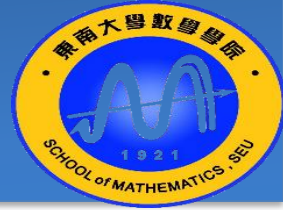
$$S[P] = \{(u, v) \in E, u \in P, v \in P^c\}$$

上述集合中的边称为图的割边。



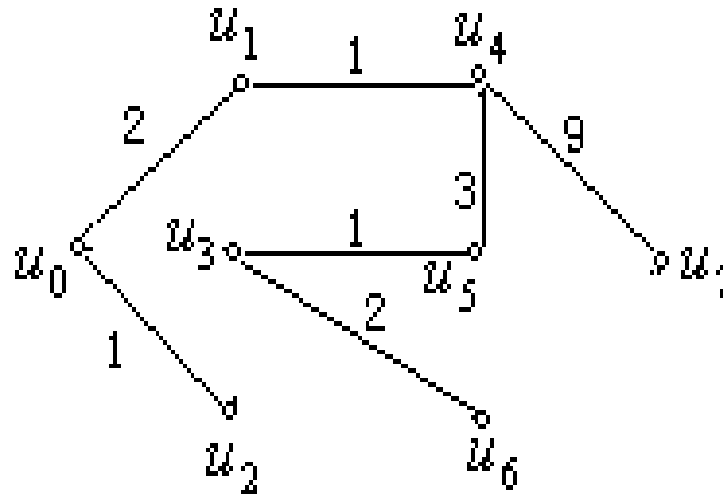
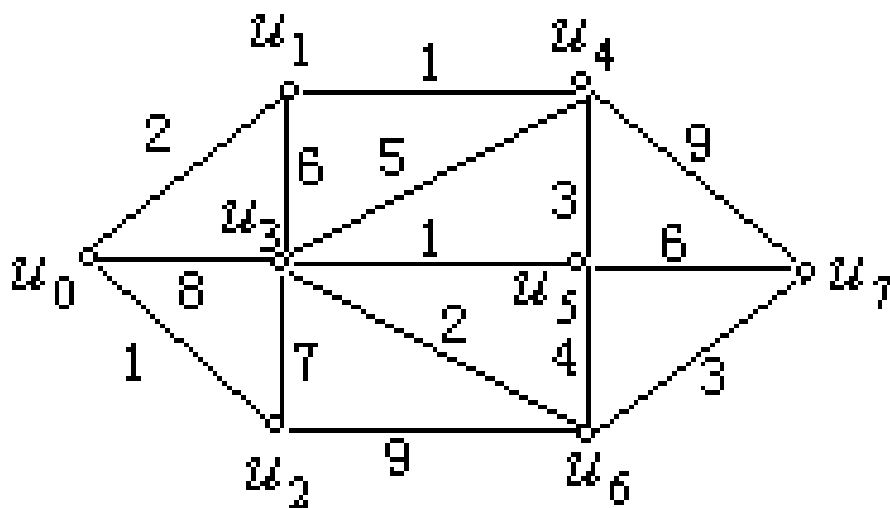


最短路问题 SRP



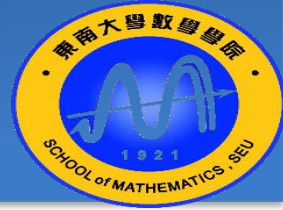
(最短路的无后效性/最优性原理) 最短路的任意一段也是相应的最短路.

如果 $P = u_1 u_2 \cdots u_k$ 是从顶点 u_1 到顶点 u_k 的最短路, 则 $P' = u_i u_{i+1} \cdots u_j$ (其中 $1 \leq i < j \leq k$) 必定是从顶点 u_i 到顶点 u_j 的最短路.



单源最短路: 求一个点到其他点的最短路

多源最短路: 求任意两个点的最短路

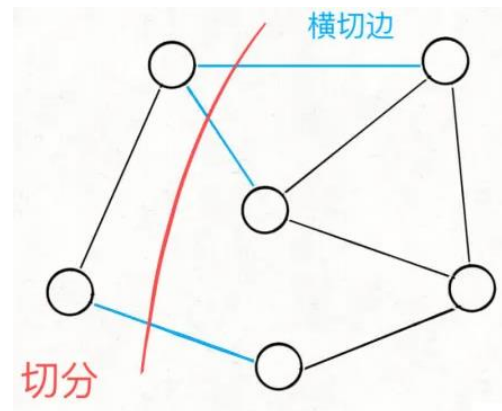


Dijkstra算法 只适用于所有边权都是正数的图

Dijkstra 算法（标号法，计算图 G 从起点 S 到终点 T 的最短路，只适用于所有边权都是正数的图）

设置集合 P 存放最短路中的顶点集， Q 存放最短路中的边，对于 P 中的点 v ，标记 $L(v)$ 表示从 S 到 v 的最短路的权。

- 1) 初始化，令 $P = \{S\}$, $Q = \emptyset$, S 点标号 $L(S) = 0$;
- 2) 对割集 $S[P]$ 中的所有边 (u, v) ，如果 $L(p) = \min_{(u,v) \in S[P]} \{L(u) + w_{uv}\}$ ，则对 p 点标号 $L(p)$ 。更新 $P = P \cup \{p\}$, $Q = Q \cup \{(u, p)\}$ (p 点可能不唯一)；
- 3) 重复第 2 步，一直到 $T \in P$ 为止。
- 4) 从 T 开始，根据 Q 中的边，反向追溯到 S ，即得最短路 $SR \subset Q$ 。



Dijkstra算法（标号法）演示

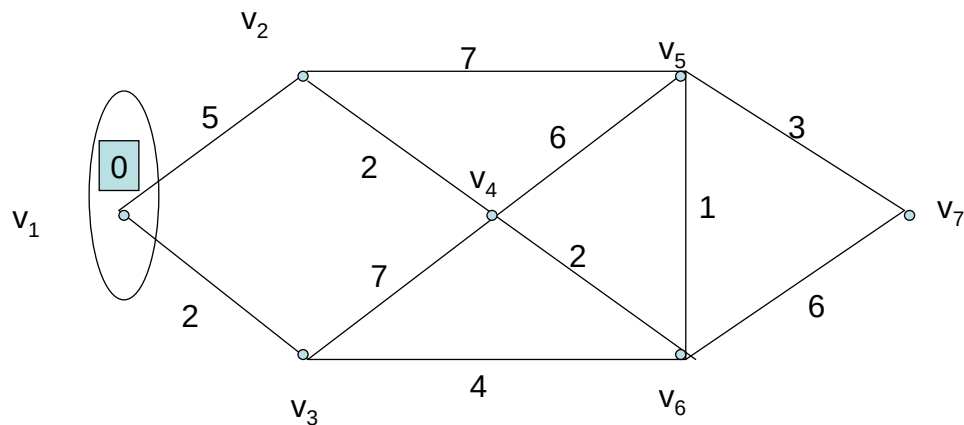


图9 (a)

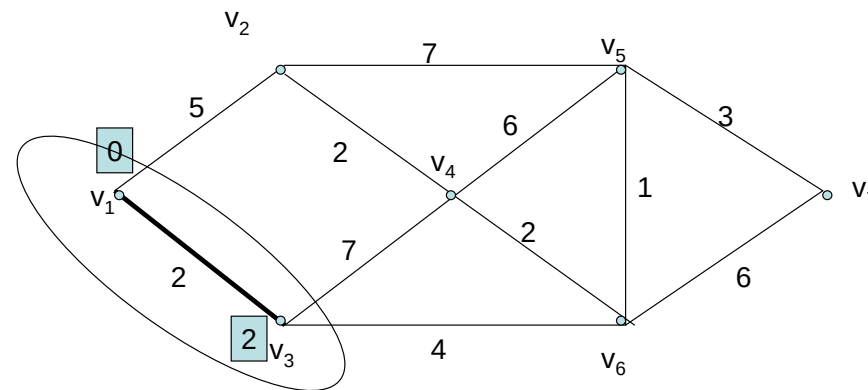


图9 (b)

Dijkstra算法（标号法）演示

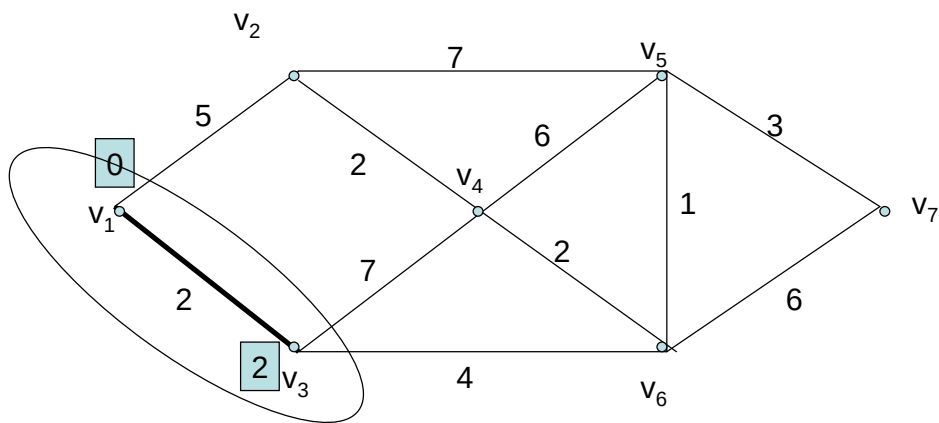


图9 (b)

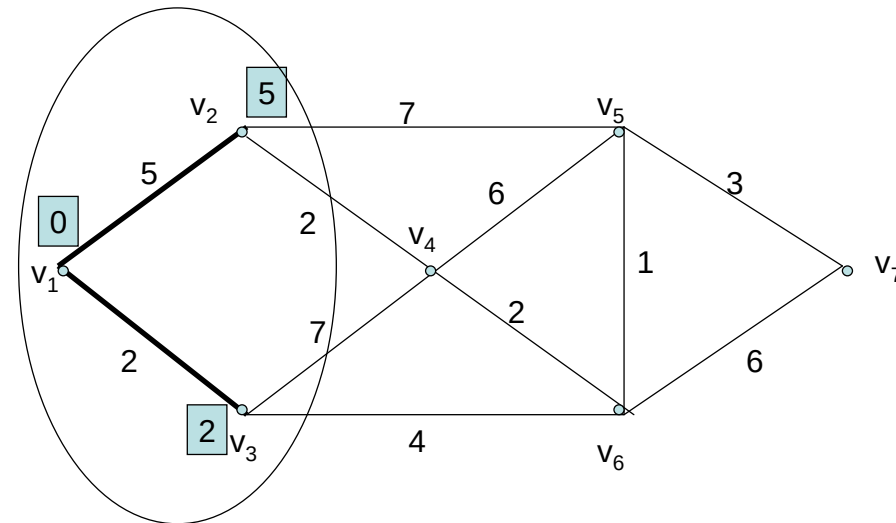


图9 (c)

Dijkstra算法（标号法）演示

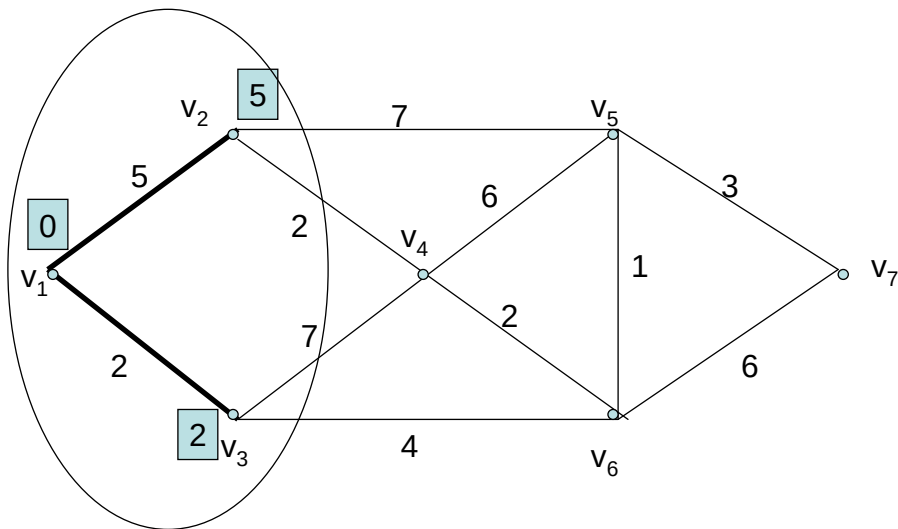


图9 (c)

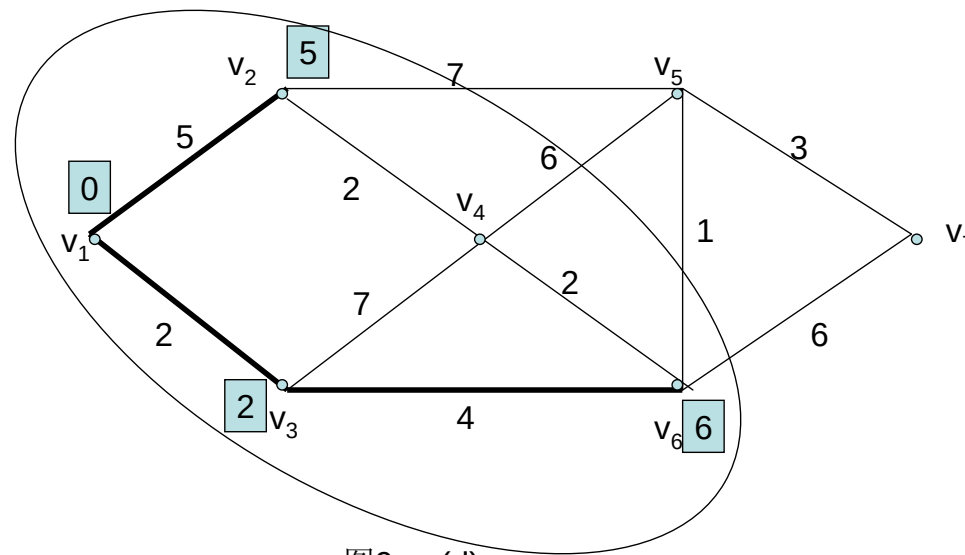


图9 (d)

Dijkstra算法（标号法）演示

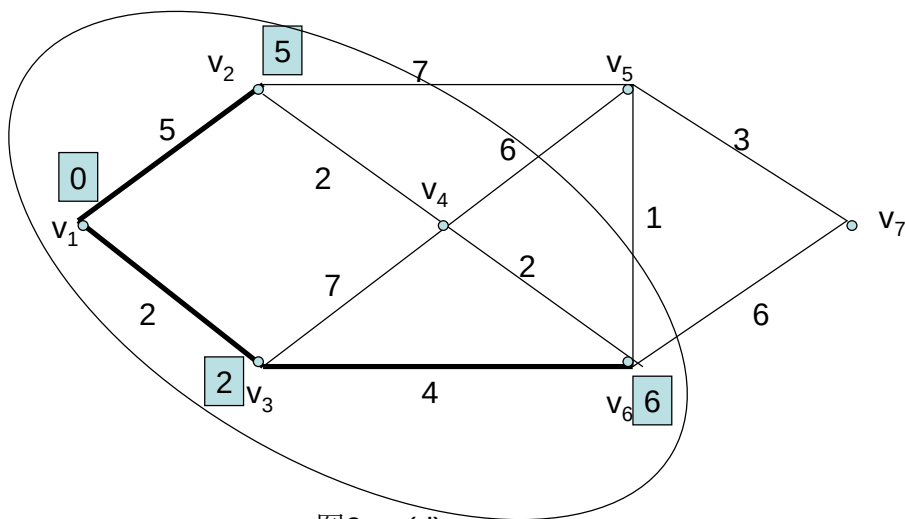


图9 (d)

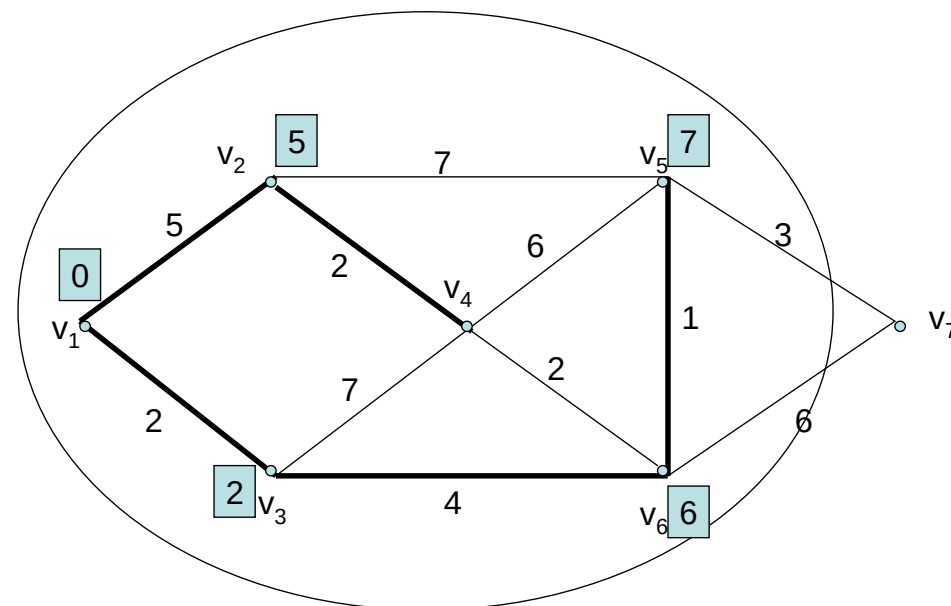


图9 (e)

Dijkstra算法（标号法）演示

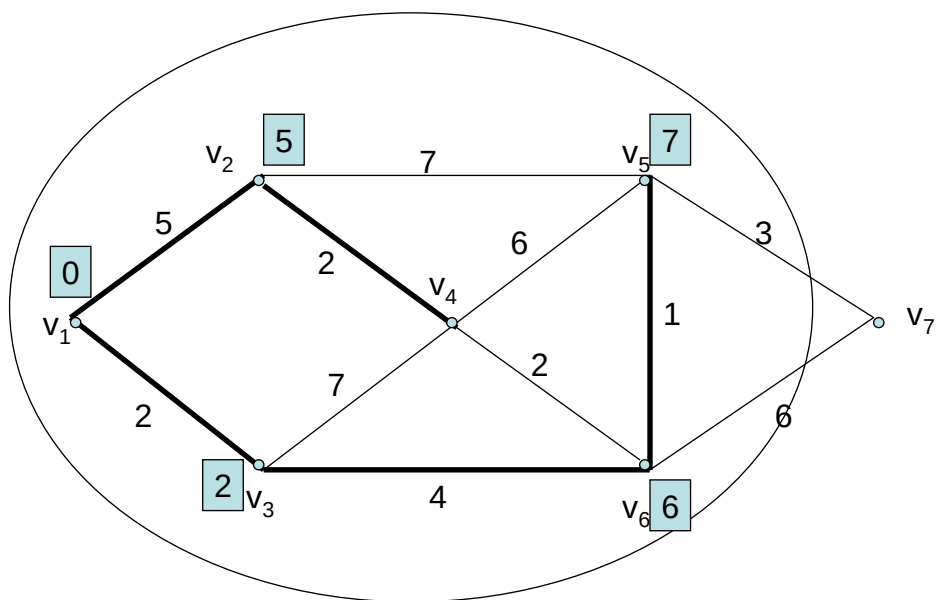


图9 (e)

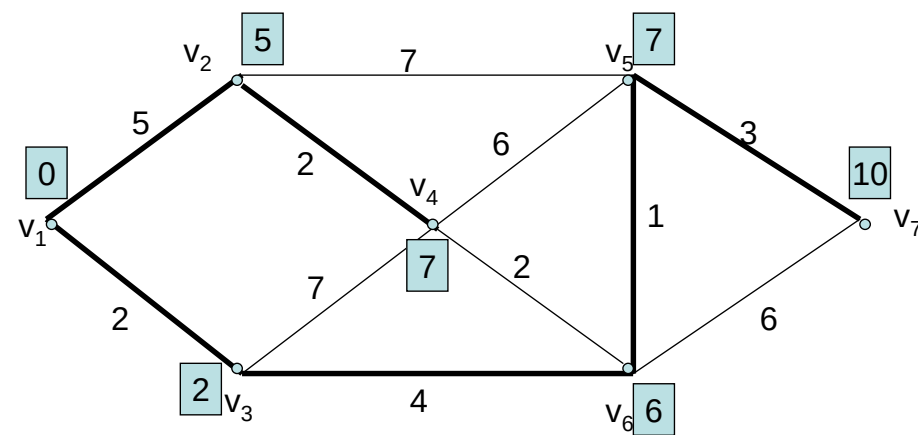
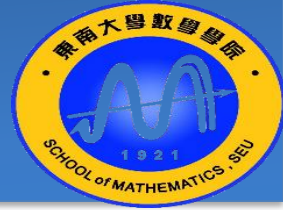


图9 (f)



最短路问题(SRP)的Lingo/Matlab解决方案 转化为0-1整数线性规划(BILP)



问题： 假设简单图有 n 个顶点，求顶点 1 到顶点 n 的最短路.

解法： 引入0-1 决策变量 $x_{ij} = \begin{cases} 1, & e_{ij} \text{ 在最短路路上} \\ 0, & \text{其它情况.} \end{cases}$

设 $W = (w_{ij})_{n \times n}$ 为赋权图邻接矩阵,特殊值
最短路取 $+\infty$ ，最长路取 $-\infty$

约束条件（出度、入度分析）：

1. 起点: $d^+(1) - d^-(1) = 1$
2. 终点: $d^+(n) - d^-(n) = -1$
3. 其它点: $d^+(i) - d^-(i) = 0$

$$\min z = \sum_{i,j=1}^n w_{ij} x_{ij}$$

$$\text{s.t. } \sum_{j=1}^n x_{j1} = 0$$

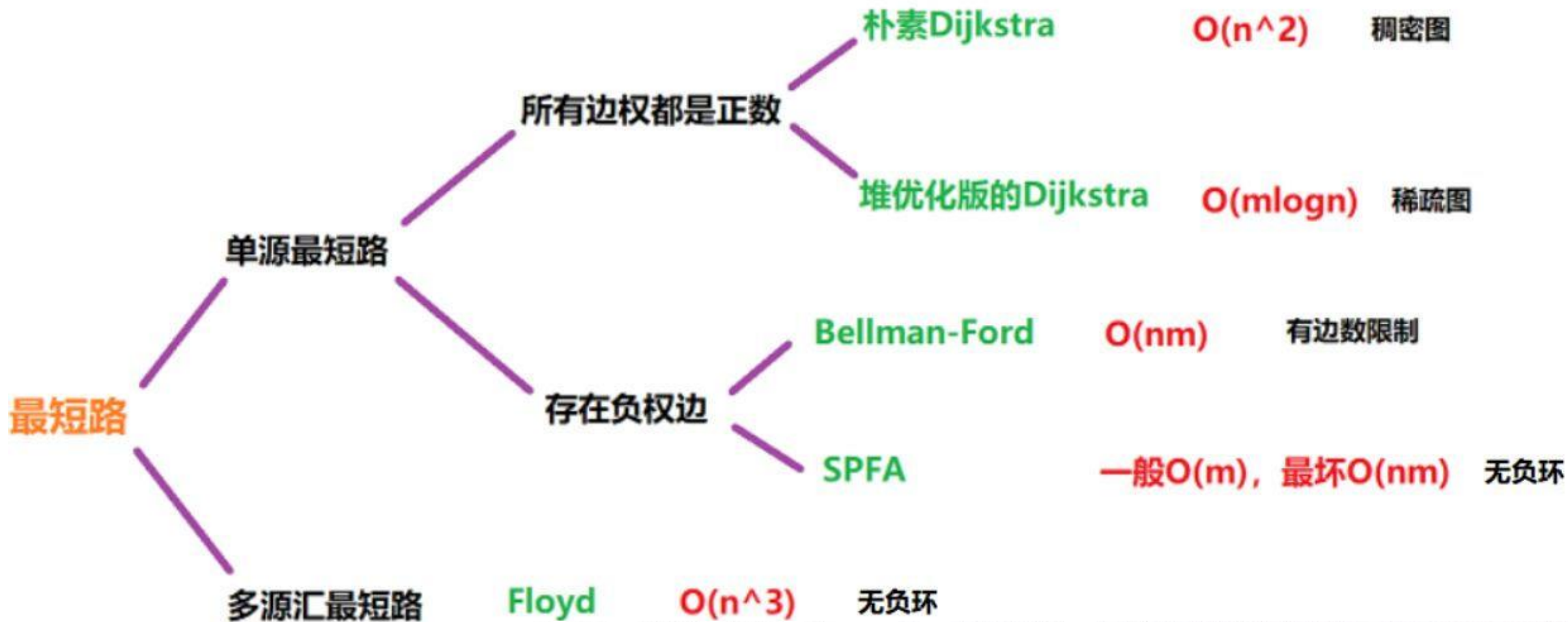
$$\sum_{j=1}^n x_{1j} = 1$$

$$\sum_{j=1}^n x_{jn} = 1$$

$$\sum_{j=1}^n x_{nj} = 0$$

$$\sum_{j=1}^n x_{ij} = \sum_{j=1}^n x_{ji}, \quad i \neq 1, n$$

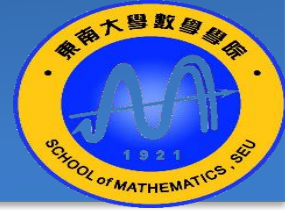
$$x_{ij} \in \{0, 1\}, i, j = 1 \sim n$$



SPFA一般效率比Bellman-Ford效率高, 但并非所有情况都可以用SPFA算法做。如果限制经过的边 $\leq k$, 则只能用Bellman-Ford做。



最小生成树问题MSTP



最小生成树问题 (MSTP) 在连通赋权图中，找一个权值最小的生成树

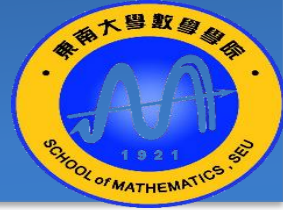
应用：电缆布线设计、网络设计、电路设计

最小生成树提供让所有点都连通，并且连通的点的总权值最小的一个方案

如果图不是连通图，则分别在每个连通分量上计算最小生成树—最小生成森林

MSTP 三种常用算法

- Kruskal 算法
- Prim 算法
- 破圈法



最小生成树的实现

Kruskal 算法 $O(m \log m)$ 思路: 从 $G = \langle V, E \rangle$ 中的**最小权重边**开始, 进行**避圈式扩**张, 逐次加边到最小生成树的边集里。

定义 Q 表示最小生成树中边的集合。

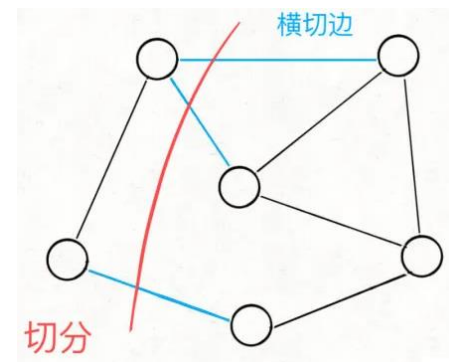
1. 初始化, $Q = \emptyset$.
2. 按照权值从小到大依次取一条边 e , 判断 $Q \cup \{e\}$ 是否**不构成圈**. 如果是则令 $Q = Q \cup \{e\}$, 否则取下一条边检验.
3. 重复步骤 2, 直到 Q 有 $n - 1$ 条边为止. 此时图 $T = \langle V, P \rangle$ 即为所求最小生成树.

特点: **Kruskal 算法时间复杂度与顶点数无关**, 较适用于点多边少的图.

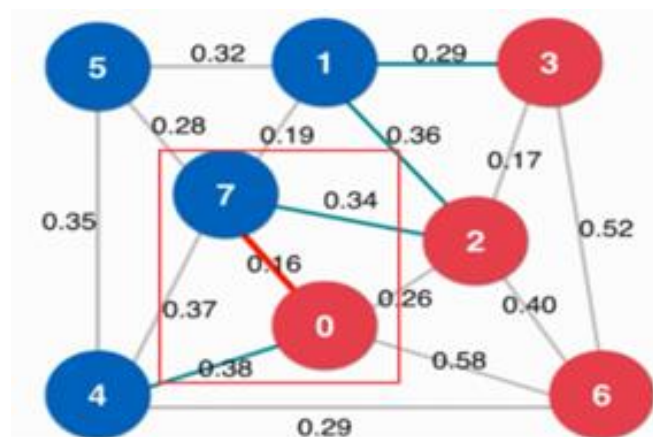
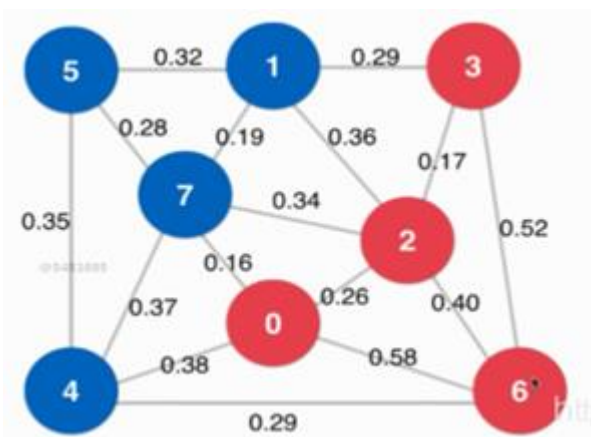
Kruskal 算法本质是贪心算法.

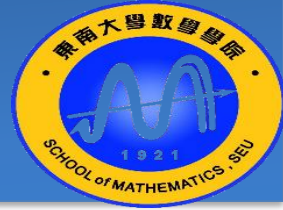
割集/切分 将一个图的顶点集 V 分成**互补且非空**的两个点集 V_1, V_1^c . 端点分别位于 V_1 和 V_1^c 的边称为**割边/横切边**, 割边的集合称为**割集/切分**

$$S[V_1] = \{(u, v) \in E(G) | u \in V_1, v \in V_1^c\}.$$



割集定理 对于任意一个割集, 其中权重最小的割边一定是构成最小生成树的一条边。





Prim 算法 $O(n^2)$

思路：从 $G = \langle V, E \rangle$ 中的任意一点开始，逐步添加权最小的割边，直到得到最小生成树。

设置集合 P 存放最小生成树中的顶点集， Q 存放最小生成树中的边。

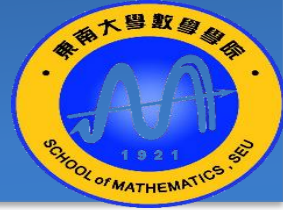
1. 取 G 中的任意一点 s 开始，令 $P = \{s\}, Q = \emptyset$ ，作切分 $S[P]$ ；
2. 寻找权最小的一条割边 $(u, v) \in S[P]$ ，做更新：

$$P = P \cup \{v\}, Q = Q \cup \{(u, v)\}$$

3. 重复步骤 2，直到 $P = V$ 为止，此时 $T = \langle P, Q \rangle$ 即最小生成树 MST。

特点：Prim 算法时间复杂度与边数无关，较适用于点少边多的稠密图。

Prime 算法本质是动态规划

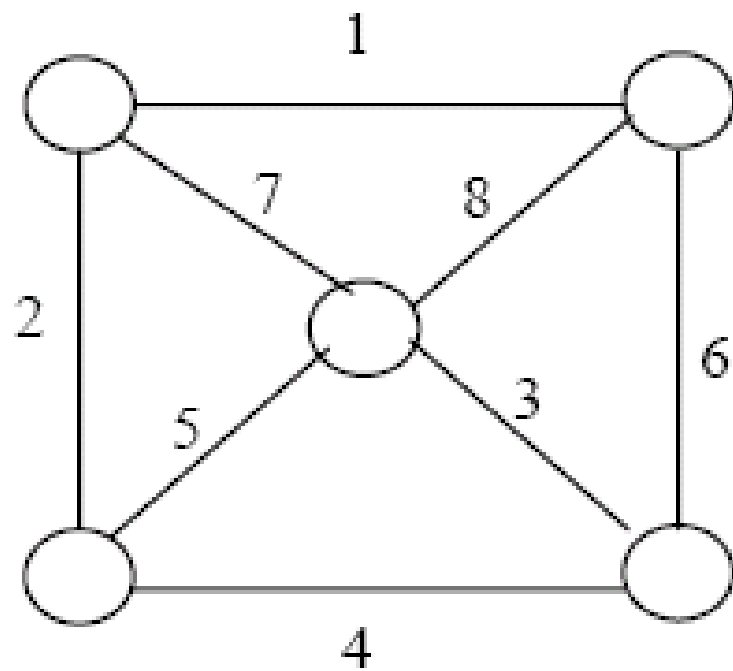


破圈法 在 Kruskal 算法基础上，我国著名数学家管梅谷教授于 1975 年提出了破圈法
核心思想：“任取连通赋权图的一个圈，去掉圈上权最大的边”，反复执行这一“破圈”步骤，直到没有圈为止。

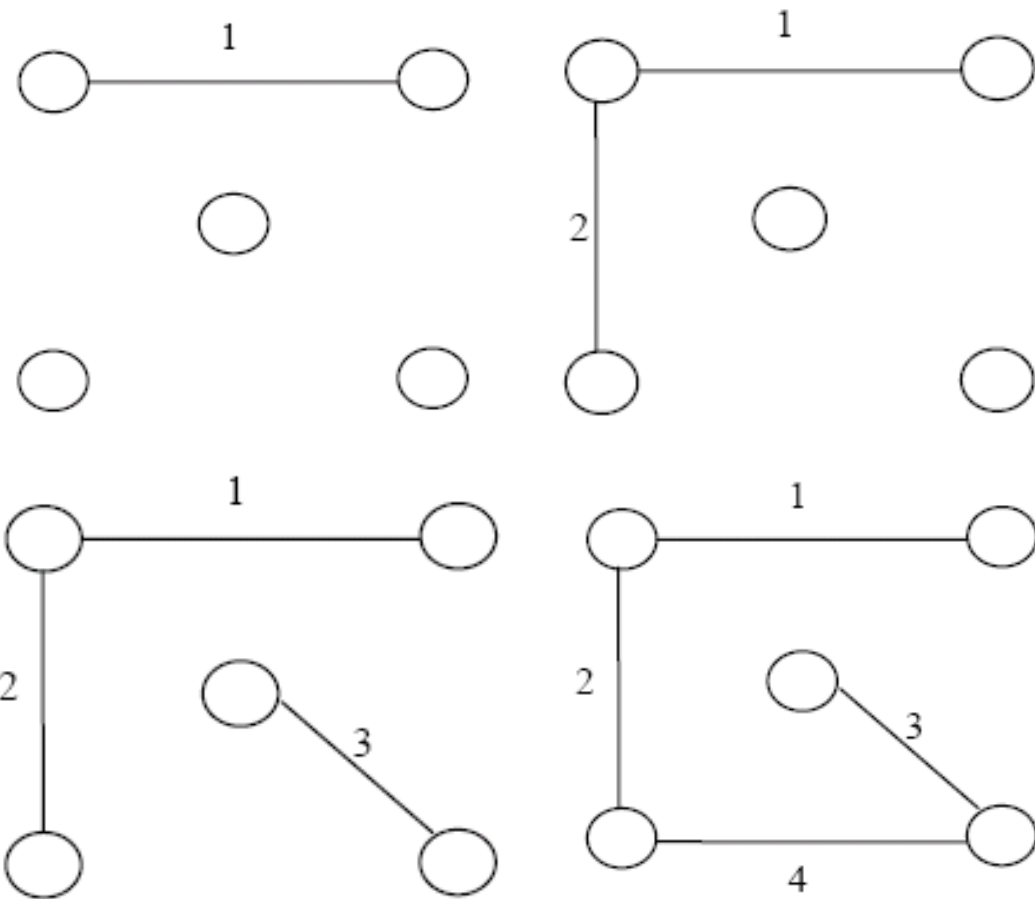
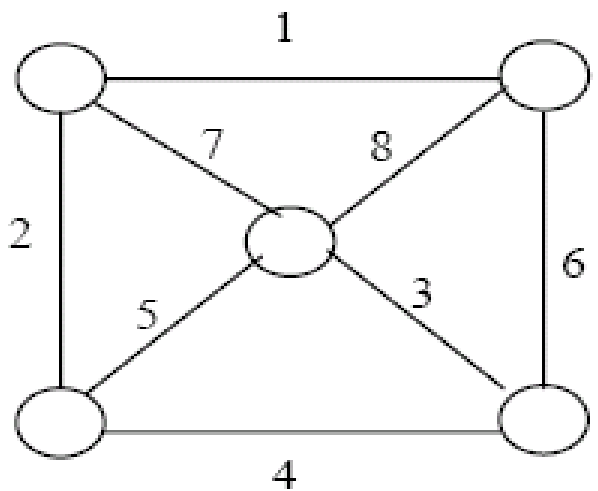
1. 按照权值从大到小依次取一条边，判断其是否位于某个圈上。如果是则将这条边删除，否则取下一条边检验。
2. 重复步骤 1，直至剩下的边数 $= n - 1$



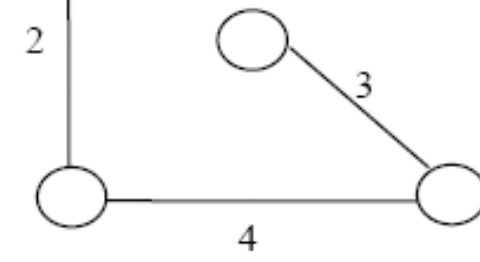
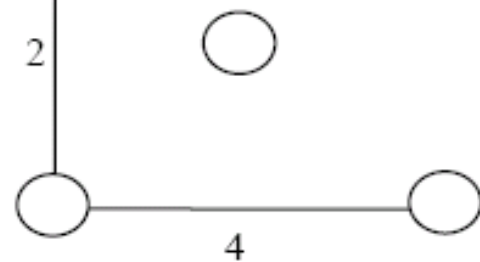
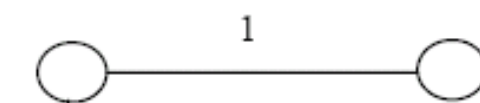
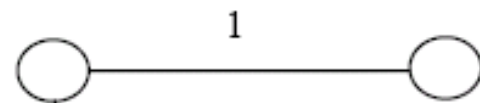
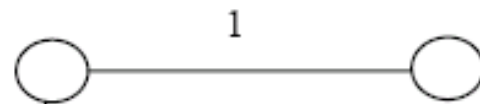
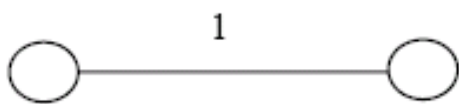
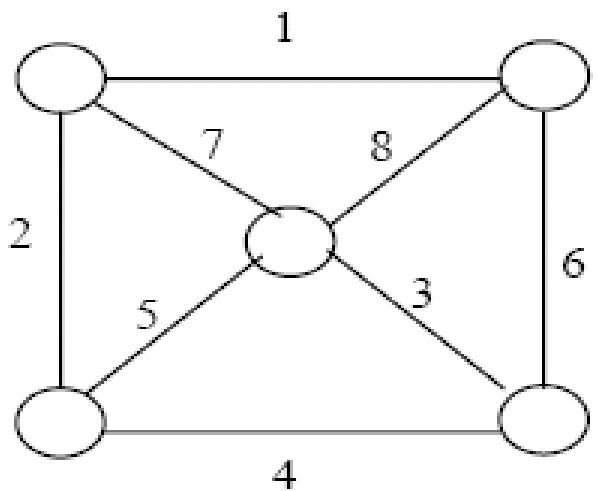
例 6.通信网络的建设问题欲建设一个连接 5 个城市的光纤通信网络。各城市间线路的造价如图所示，求一个总造价最少的线路建设方案。



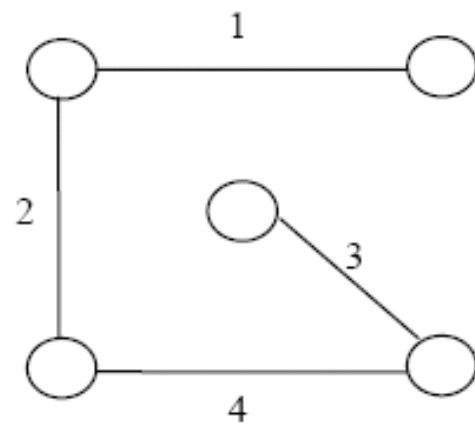
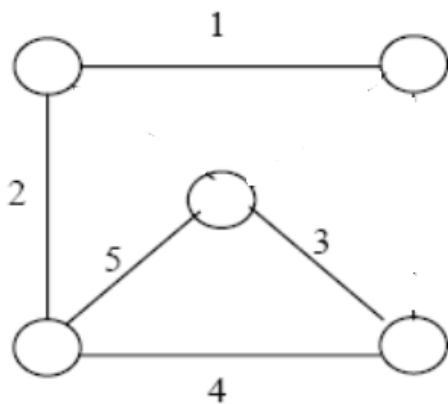
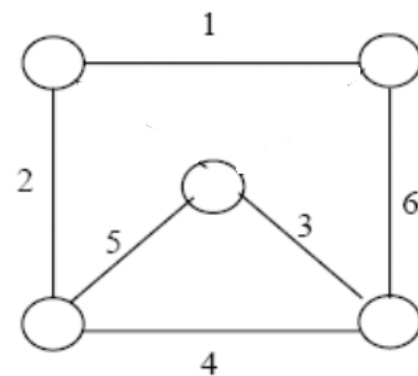
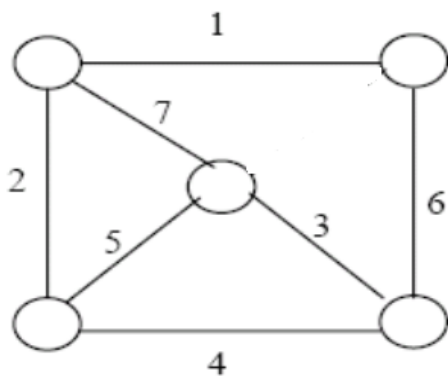
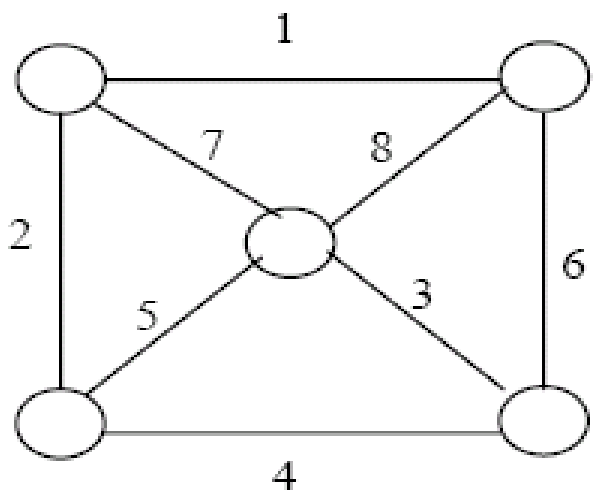
解法1: Kruskal算法



解法2: Prim算法

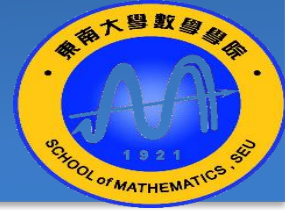


解法3：管梅谷破圈算法





最小生成树问题(MSTP)的数学模型(一)



问题: 假设无向简单图 $G = \langle V, E \rangle$ 有 n 个顶点, m 条边, 求最大生成树。

基于边权向量的模型: 对于边 $e_i \in E$, 引入 **0-1 决策变量** $x_i = \begin{cases} 1, & e_i \text{ 被选中} \\ 0, & e_i \text{ 其他情况.} \end{cases}$

设 $W = (w_1, w_2, \dots, w_m)$ 为边权向量

BILP 模型

约束条件:

1. MST 的边数 $= n - 1$
2. (**子图无回路约束**) 任何非空真子图的边数不超过其顶点数减 1,
即由 V 的任意非空真子集 S 生成的导出子图 G_S 的边数

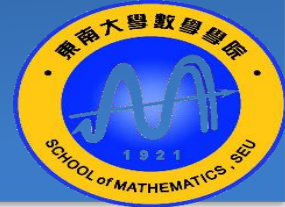
$$m(G_S) \leq |S| - 1$$

$|S|$ 为 S 的顶点数

$$\begin{aligned} \min z &= \sum_{i=1}^m w_i x_i \\ s.t. \quad &\sum_{i=1}^m x_i = n - 1 \\ &\sum_{e_i \in G_S} x_i \leq |S| - 1, \forall S \subset V, 2 \leq |S| \leq n - 1 \\ &x_i \in \{0, 1\}, i = 1 \sim m \end{aligned}$$



最小生成树问题(MSTP)的Lingo/Matlab解决方案 转化为整数线性规划(ILP)



问题： 假设图有 n 个顶点，设顶点 1 为生成树的根，求最小生成树。

解法： 引入

1. **0-1 决策变量** $x_{ij} = \begin{cases} 1, & e_{ij} \text{在最小生成树上} \\ 0, & \text{其它情况.} \end{cases}$

2. 层变量 l : 如果第 i 个顶点位于树的第 k 层，则 $l_i = k$

设 $W = (w_{ij})_{n \times n}$ 为赋权图邻接矩阵, **特殊值**

最小生成树取 $+\infty$ ，最大生成树取 $-\infty$

约束条件：

1. 根の入度为 0，出度不小于 1
2. 根以外的每个顶点入度为 1
3. **避圈**

$$\min z = \sum_{i,j=1}^n w_{ij} x_{ij}$$

$$s.t. \sum_{j=1}^n x_{1j} \geq 1$$

$$\sum_{j=1}^n x_{j1} = 0$$

$$\sum_{j=1}^n x_{ji} = 1, i \neq 1$$

$$l_1 = 0$$

$$2 - x_{1i} \leq l_i \leq n - 1 - (n - 2)x_{1i}, i > 1$$

$$l_j \geq l_i + (n - 1)x_{ij} + (n - 3)x_{ji} - (n - 2), i, j > 1, j \neq i$$

$$x_{ij} \in \{0, 1\}, l_i \in N, l_i \in [0, n - 1]$$



LINGO求解MSTP



2. LINGO 求解

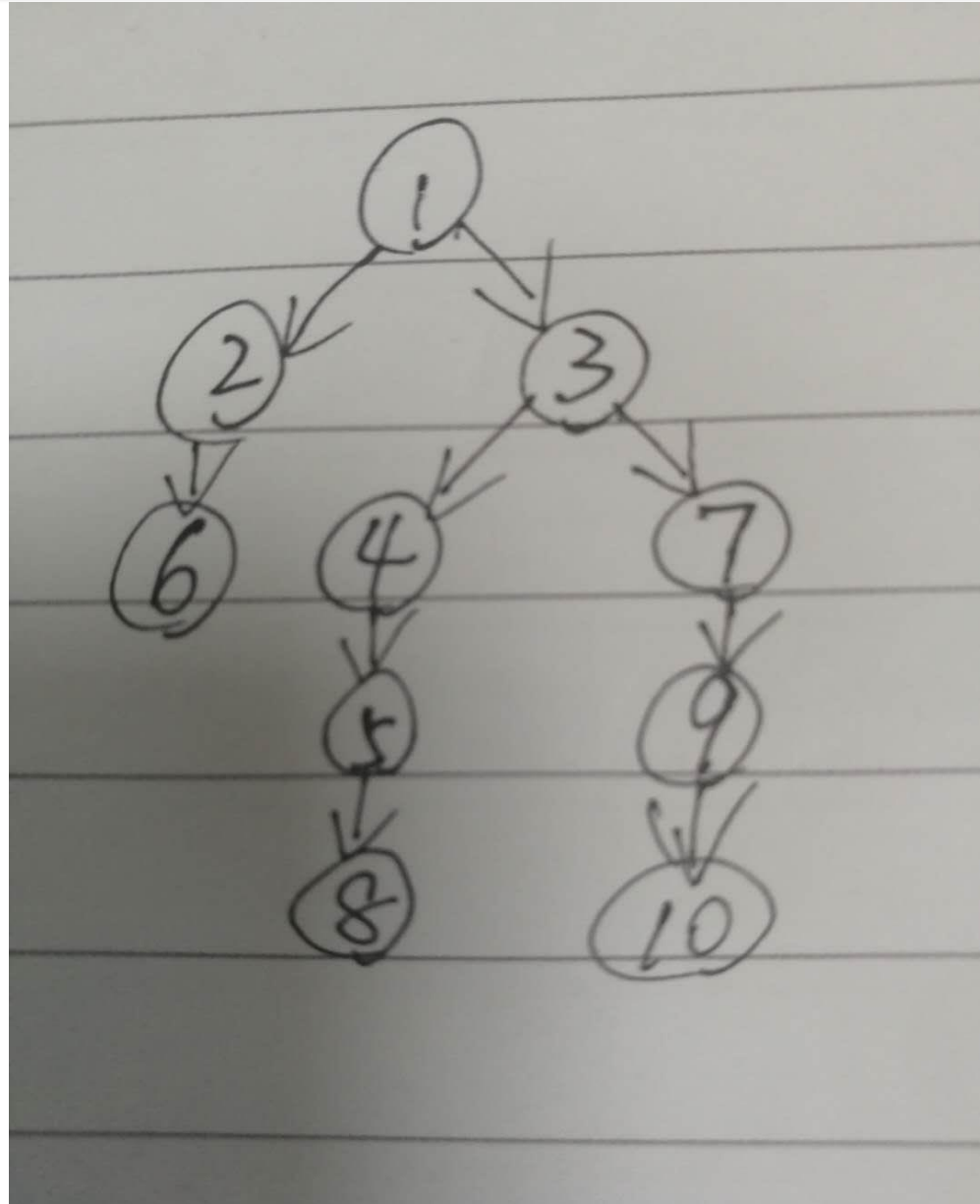
model:

```
sets:
cities /1..10/: level;
link(cities , cities): distance , x;
endsets
data:
! Distance matrix need not be ;
! symmetric;
distance=0 8 5 9 12 14 12 16 17 22
8 0 9 15 16 8 11 18 14 22
5 9 0 7 9 11 7 12 12 17
9 15 7 0 3 17 10 7 15 15
12 16 9 3 0 8 10 6 15 15
14 8 11 17 8 0 9 14 8 16
12 11 7 10 10 9 0 8 6 11
16 18 12 7 6 14 8 0 11 11
17 14 12 15 15 8 6 11 0 10
22 22 17 15 15 16 11 11 10 0;
enddata
n=@size(cities);
```



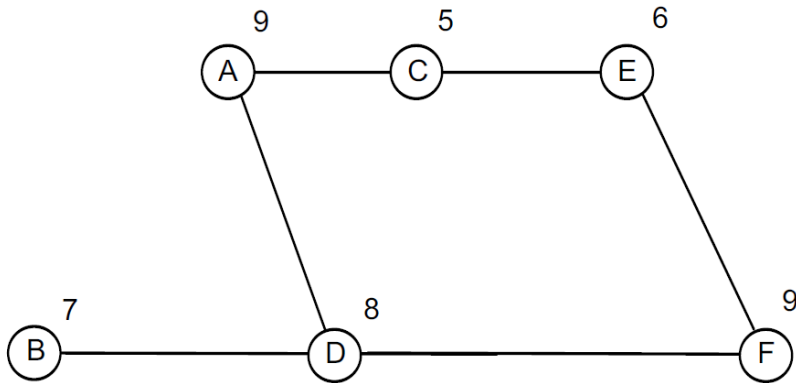
```
min=@sum(link(i,j)|i#ne#j: distance(i,j)*x(i,j));
@sum(cities(i)|i#gt#1:x(i,1))=0;
@sum(cities(i)|i#gt#1:x(1,i))>=1;

level(1)=0;
@for(cities(i)|i#gt#1:
    level(i)<=n-1-(n-2)*x(1,i);
    level(i)>=2-x(1,i);
    @sum(cities(j)|j#ne#i:x(j,i))=1;
    @for(cities(j)|j#gt#1 #and# j#ne#i:
        level(j)>=level(i)+(n-1)*x(i,j)+(n-3)*x(j,i)-(n-2););
@for(cities: @bin(x));
@bnd(1,level,n-1);
);
```



Activity-on-Node(AON) vs. Activity-on-Arc(AOA) Network Diagrams

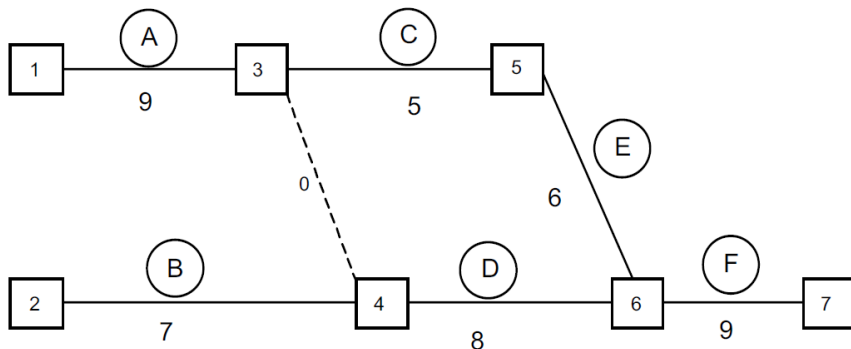
Figure 8.3 An Activity-on-Node Representation



AON

- Each activity is represented by a node in the network.
- A precedence relationship between two activities is represented by an arc or link between the two.
- AON may be less error prone because it does not need dummy activities or arcs.

Figure 8.4 An Activity-on-Arc Representation

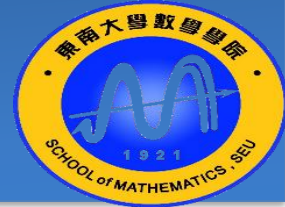


AOA

- Each activity is represented by an arc in the network.
- If activity X must precede activity Y , there are X leads into arc Y . The nodes thus represent events or “milestones” (e.g., “finished activity X ”). Dummy activities of zero length may be required to properly represent precedence relationships.
- AOA historically has been more popular, perhaps because of its similarity to Gantt charts used in scheduling.



Program Evaluation and Review Technique (PERT) and Critical Path Method (CPM)



PERT是利用网络分析制定计划以及对计划予以评价的技术。

应用范围

PERT 网络来表示一项复杂工程，它含有各个子工程之间的优先关系及各个子工程的持续时间等信息。

关心的主要问题：整个工程完成的需要的最短时间是多少；哪些事件的延期将会影响整个工程的进度，而加速这些事件是否会提高整个工程的效率。

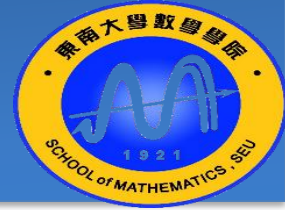
因此，通常在 PERT 网中列出完成预定工程计划所需要进行的事件（子工程）及状态（里程碑），每个事件计划完成的时间，事件与状态之间的关系，从而可以确定该项工程是否可行，估算工程完成的时间以及确定哪些事件是影响工程进度的关键。

构造 PERT 图，需要明确四个概念：事件、标志状态、松弛时间和关键路径。

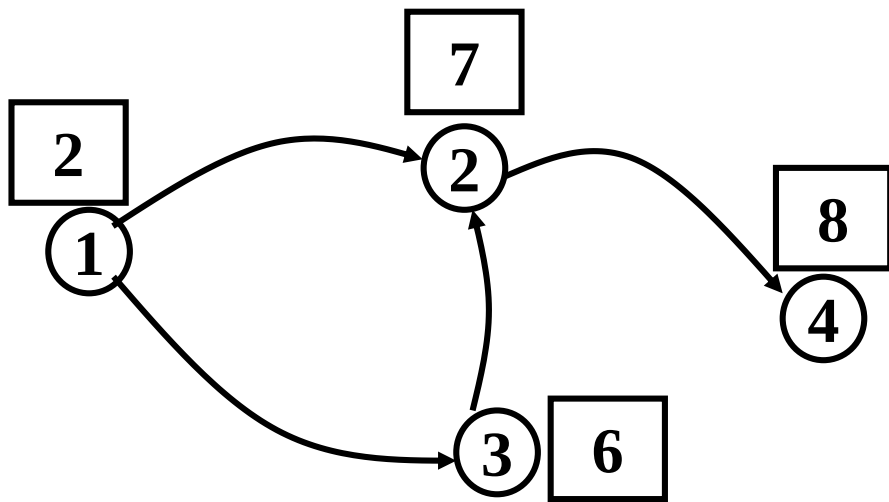
1. **状态**（里程碑，Events）：表示事件结束的那一时刻；
2. **事件**（Activities）：表示从一个状态到另一个状态之间的子工程；
3. **松弛时间**（Slack time）：不影响完工前提下事件可能被推迟完成的最大时间；
4. **关键路径**（Critical Path）：是 PERT 网络中花费时间最长的状态、事件的序列。



Program Evaluation and Review Technique (PERT) and Critical Path Method (CPM)



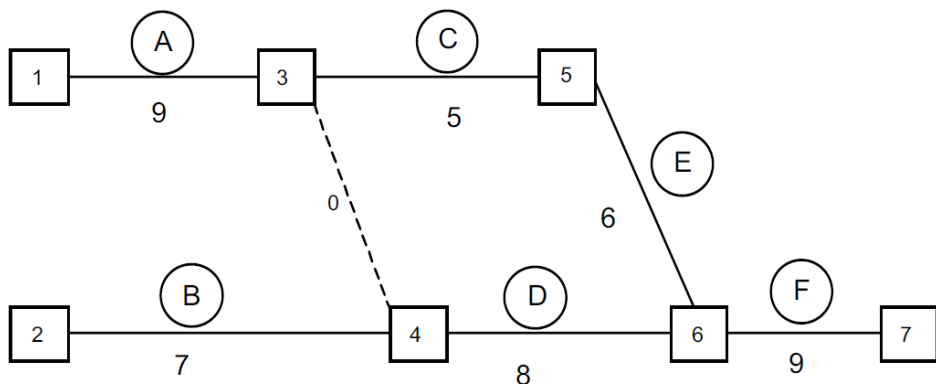
- 对于**简单图**，可以采用 AON 记号，项目由多个事件（顶点）组成，每个事件需要一定时间（点的权值）完成，某个事件发生意味着其所有**前导事件**（predecessors，该顶点的父节点）的完成，每个事件都是其先导事件的**后继事件**（successors，子节点）。求完成这个项目的最短时间。
- 开工事件与竣工事件。
- 关键路径 (CP)**：在 PERT 网络中**花费时间最长的事件序列**。



关键路径特点：

- PERT 图中路的权定义为路经过的所有顶点的权值和。
- 关键路径对应开工事件到竣工事件的一条最长路**
- 工程的**最短工期就是关键路径的权**
- 开工、竣工事件一定在 CP 上；
- 除开工事件外，仅当有一个前导事件在 CP 上时该事件才有可能出现在 CP 上；
- 出现在 CP 上的一个事件如果有后继事件，那么只有一个后继事件出现在 CP 上。

Figure 8.4 An Activity-on-Arc Representation

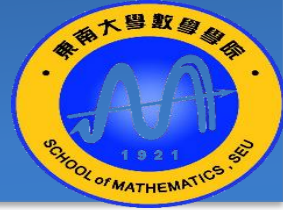


AOA 网络 可以有重边，有向边表示事件，顶点表示事件状态，起点表示事件开始状态，终点表示事件结束状态.

1. 进入一条边的起点的所有边是该边代表的事件的前导事件，所有前导事件全部结束，该事件才能开始。
2. 从一条边的终点出发的所有边是该边代表的事件的后续事件，只有当该事件结束，其后续事件才能开始。
3. 一个 PERT 网络只有一个入度 $=0$ 的顶点，表示工程开工，只有一个出度 $=0$ 的顶点，表示工程竣工。
4. 关键路径：从开工到竣工具有最大权 (长度) 的路径叫关键路径 CP。
5. 关键事件：在关键路径 CP 上的事件叫关键事件。



AON网络CPM关键路径问题的Lingo/Matlab解决方案 转化为0-1整数线性规划(BILP)



问题： 假设 PERT 图是用 AON 表示的 n 阶简单图，顶点 1(开工事件)，顶点 n (竣工事件)。求关键路径。

解法一： 求解最长路得到 CP。

引入0-1决策变量 $x_{ij} = \begin{cases} 1, & e_{ij} \text{ 在 CP 上} \\ 0, & \text{其它情况.} \end{cases}$

设 $W = (w_{ij})_{n \times n}$ 为赋权图邻接矩阵,特殊值取 $-\infty$

算法特点:

- (1)图没有重边
- (2)决策变量为0-1矩阵
- (3)输入参数为赋权图邻接矩阵
- (4)与最短路邻接矩阵不同

约束条件（出度、入度分析）：

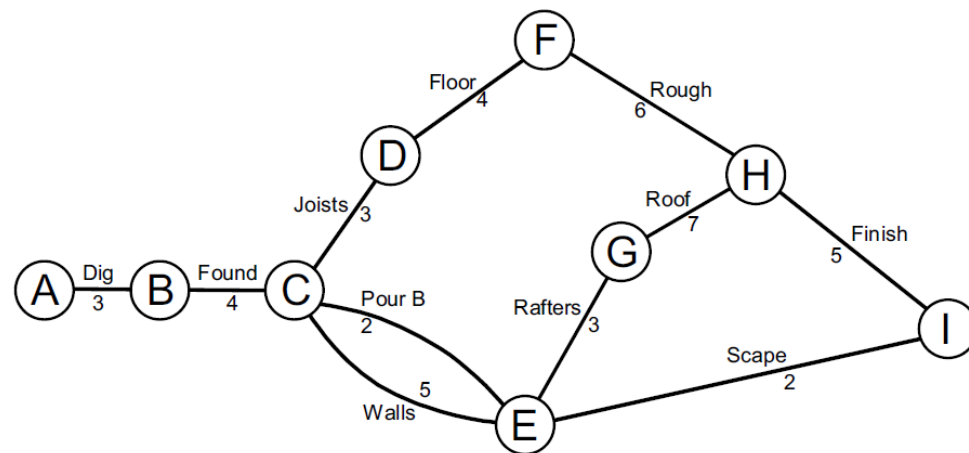
1. 起点: $d^+(1) = 1, d^-(1) = 0$
2. 终点: $d^+(n) = 0, d^-(n) = 1$
3. 其它点: $d^+(i) - d^-(i) = 0$

$$\begin{aligned} \max z &= \sum_{i,j=1}^n w_{ij} x_{ij} \\ \text{s.t. } &\sum_{j=1}^n x_{j1} = 0 \\ &\sum_{j=1}^n x_{1j} = 1 \\ &\sum_{j=1}^n x_{jn} = 1 \\ &\sum_{j=1}^n x_{nj} = 0 \\ &\sum_{j=1}^n x_{ij} - \sum_{j=1}^n x_{ji} = 0, \quad i \neq 1, n \\ &x_{ij} \in \{0, 1\}, i, j = 1 \sim n \end{aligned}$$

例 1. 对下表表示的工程建立合适的 PERT 图

Activity	Mnemonic	Activity Time	Predecessors (Mnemonic)
Dig Basement	DIG	3	—
Pour Foundation	FOUND	4	DIG
Pour Basement Floor	POURB	2	FOUND
Install Floor Joists	JOISTS	3	FOUND
Install Walls	WALLS	5	FOUND
Install Rafters	RAFTERS	3	WALLS, POURB
Install Flooring	FLOOR	4	JOISTS
Rough Interior	ROUGH	6	FLOOR
Install Roof	ROOF	7	RAFTERS
Finish Interior	FINISH	5	ROUGH, ROOF
Landscape	SCAPE	2	POURB, WALLS

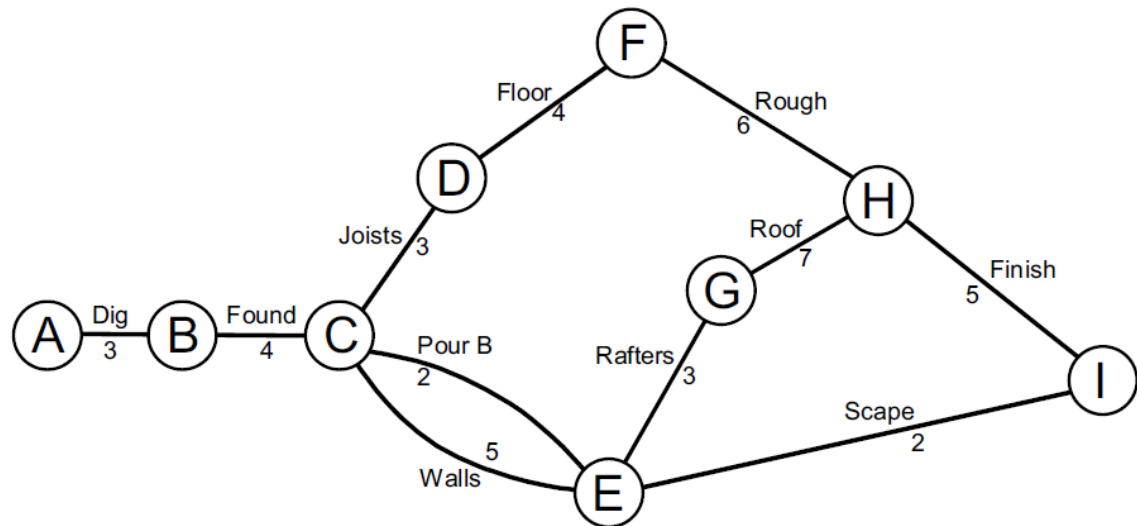
Figure 8.2 Activity-on-Arc PERT/CPM Network



Activity	Mnemonic	Activity Time	Predecessors (Mnemonic)
1 Dig Basement	DIG	3	—
2 Pour Foundation	FOUND	4	DIG
3 Pour Basement Floor	POURB	2	FOUND
4 Install Floor Joists	JOISTS	3	FOUND
5 Install Walls	WALLS	5	FOUND
6 Install Rafters	RAFTERS	3	WALLS, POURB
7 Install Flooring	FLOOR	4	JOISTS
8 Rough Interior	ROUGH	6	FLOOR
9 Install Roof	ROOF	7	RAFTERS
10 Finish Interior	FINISH	5	ROUGH, ROOF
11 Landscape	SCAPE	2	POURB, WALLS

解: 因为存在重边, 考虑采用 AOA 形式构建 PERT 图, 用关联矩阵表示

Figure 8.2 Activity-on-Arc PERT/CPM Network



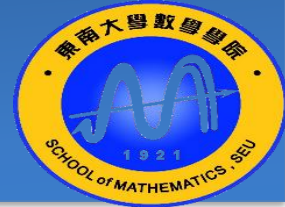
采用关联矩阵，优点如下：

1. 关联矩阵可以表示带重边的图
2. 关联矩阵可以表示出度、入度
3. 关联矩阵可以表示事件关联的两个状态的先后顺序

$$\begin{pmatrix} 1 & & & & & & & & & \\ -1 & 1 & & & & & & & & \\ & -1 & 1 & 1 & 1 & & & & & \\ & & & -1 & & & 1 & & & \\ & & & & -1 & 1 & & & & \\ & -1 & & & -1 & 1 & & & & 1 \\ & & & & & & -1 & 1 & & \\ & & & & & & & & 1 & \\ & & & & & -1 & & & & \\ & & & & & & 1 & & & \\ & & & & & & & -1 & -1 & 1 \\ & & & & & & & & -1 & -1 \end{pmatrix}$$



AOA网络PERT/CPM的Lingo/Matlab解决方案 转化为线性规划(LP)



基本问题： 求整个工程的最早完工时间.

设顶点 1 和顶点 n 分别标志开工和竣工

解法二： 设 S_i 是状态 i 的最早出现时间, t_{ij} 是事件 (i, j) 的计划（正常）消耗时间。

$$\min z = S_n$$

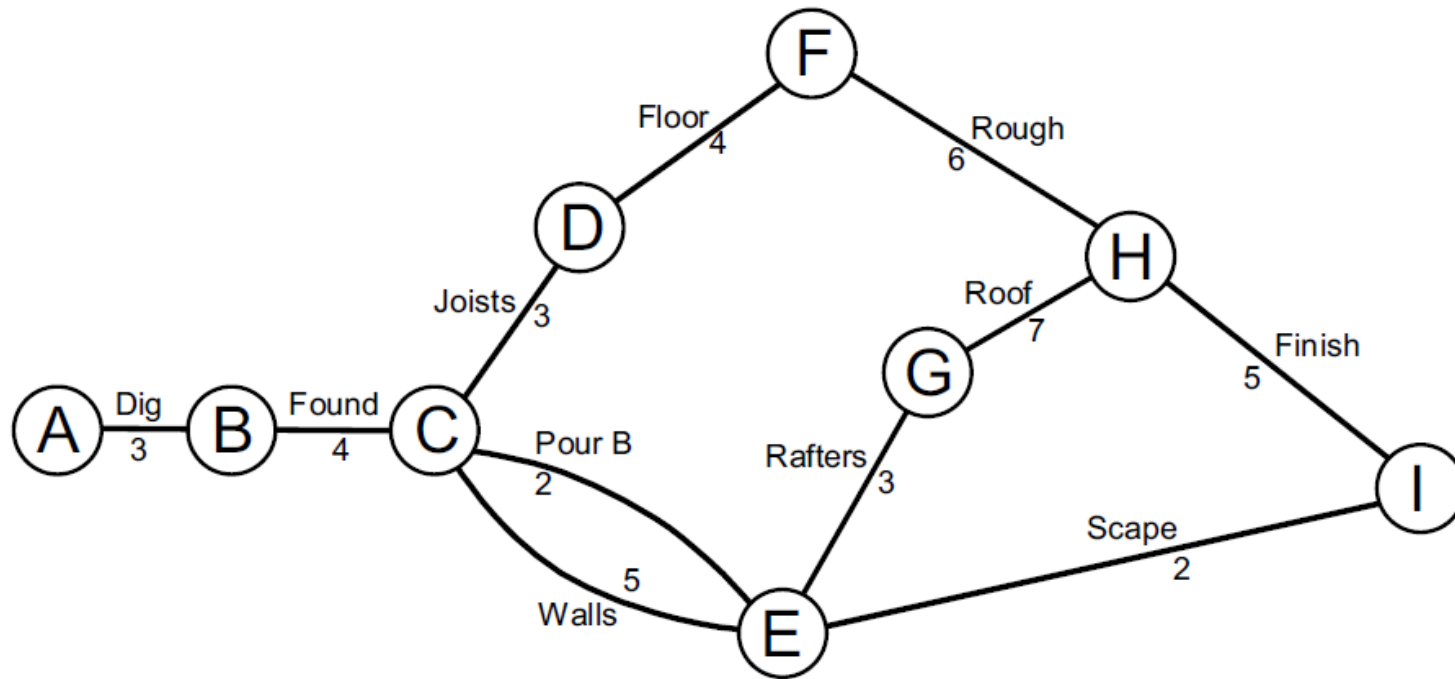
$$s.t. S_j - S_i \geq t_{ij}, \forall e_{i,j} \in E$$

$$S_i \geq 0, i \in V$$

$$\min z = S_n - S_1$$

$$s.t. S_j - S_i \geq t_{ij}, \forall e_{i,j} \in E$$

Figure 8.2 Activity-on-Arc PERT/CPM Network



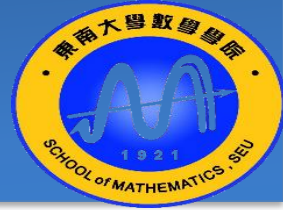
Listing 1: Example Lingo Code for CP

```

1 # This is a comment
2 MAX = I-A;
3
4 B-A >= 3; ! DIG;
5 C-B >= 4; ! FOUND;
6 E-C >= 2; .
7 D-C >= 3; .
8 E-C >= 5; .
9 F-D >= 4;
10 G-E >= 3;
11 H-F >= 6;
12 H-G >= 7;
13 I-H >= 5;
14 I-E >= 2;
  
```



AOA网络CPM关键路径问题的Lingo/Matlab解决方案 转化为0-1整数线性规划(BILP)



问题: 假设 PERT 图有 m 个事件 (AOA), n 个顶点, 1 号顶点表示开工, n 号顶点标志竣工, 求关键路径 CP。

解法三: 对于第 i 条边, 定义它的前导事件集 P_i , 后续事件集 S_i .

对每个事件 (边) 引入 **0-1 决策变量** $x_j = \begin{cases} 1, & e_j \text{ 在 CP 上} \\ 0, & \text{其它情况.} \end{cases}$

设 $p = (t_1, t_2, \dots, t_m)^T \in R^m$ 为事件**权向量**

算法特点:

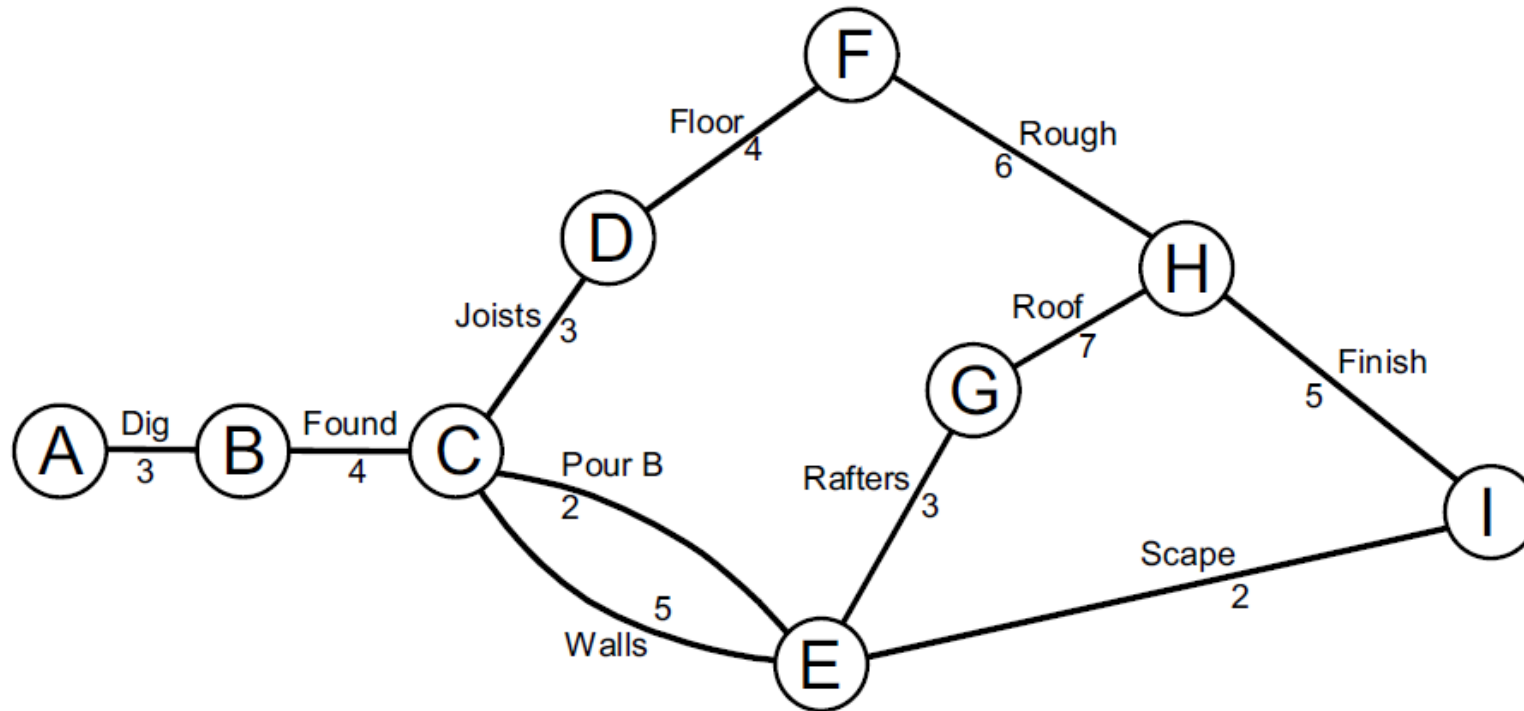
- (1)图可以有重边
- (2)决策变量为向量
- (3)输入参数为赋权图权向量

$$\max z = \sum_{j=1}^m p_j x_j$$

$$st: \sum_{j \in S_i} x_j - \sum_{j \in P_i} x_j = \begin{cases} 1, & i = 1 \\ -1, & i = n \\ 0, & i \neq 1, n \end{cases}$$

$$x_j \in \{0, 1\}, j = 1 \sim m$$

Figure 8.2 Activity-on-Arc PERT/CPM Network



Listing 1: Example Lingo Code for CP

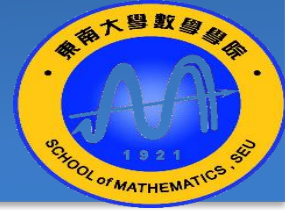
```

1 # This is a comment
2 MAX = 3*DIG +4*FOUND +2*POURB +3*JOISTS
      +5*WALLS +3*RAFTERS +4*FLOOR +6*ROUGH
      +7*ROOF + 5*FINISH +2*SCAPE;
3
4 DIG =1;
5 FOUND -DIG= 0;
6 JOISTS +POURB +WALLS -FOUND = 0;
7 FLOOR -JOISTS = 0;
8 RAFTERS +SCAPE -POURB -WALLS = 0;
9 ROUGH -FLOOR = 0;
10 ROOF -RAFTERS = 0;
11 FINISH -ROUGH -ROOF = 0;
12 -FINISH -SCAPE = -1;
  
```



Crashing Analysis的Lingo/Matlab解决方案

转化为线性规划(LP) AON



问题 求符合规定完成时间 F 而费用最小的突击进度可行计划

$$\begin{aligned} \min C &= \sum_{i,j=1}^n p_{ij} r_{ij} \\ s.t. \quad & S_j - S_i \geq T_{ij} - r_{ij}, \forall e_{i,j} \in E \\ & S_n - S_1 \leq F \\ & 0 \leq r_{ij} \leq R_{ij}, \forall e_{ij} \in E \end{aligned}$$

- S_i : 状态 i 最早开工时间
- T_{ij} : 事件 (i, j) 正常耗时
- R_{ij} : 事件 (i, j) 最多压缩时间
- r_{ij} : 事件 (i, j) 实际压缩时间
- p_{ij} : 事件 (i, j) 突击边际费用
- F : 项目预期（规定）完工时间



例8.5调度问题



例 2. 工程施工调度问题 (教材例 8.5)

表 8.1 体育馆施工数据

任务	描述	耗时	先决条件	最大缩短时间	每周额外开支
1	工地布置	2		0	
2	场地平整	16	1	3	30
3	打地基	9	2	1	26
4	道路网管	8	2	2	12
5	底层施工	10	3	2	17
6	主场地施工	6	4,5	1	15
7	更衣室分隔	2	4	1	8
8	看台布置	2	6	0	
9	顶部施工	9	4,6	2	42
10	照明	5	4	1	21
11	阶梯安装	3	6	1	18
12	封顶	2	9	0	
13	更衣室布置	1	7	0	
14	售票处施工	7	2	2	22
15	第二通道	4	4,14	2	12
16	信号设施	3	8,11,14	1	6
17	草坪	9	12	3	16
18	交付使用	1	17	0	

引入变量 S_i 表示任务 i 最早开始的时间. 唯一的约束条件是任务的先后顺序. 只有在其先决任务都完成后, 任务 j 才可以进行. 如果在 i 和 j 之间有一条弧相连, 则任务 i 的完成时间必须不大于任务 j 的开始时间, 即

$$S_i + T_i \leq S_j, \quad \forall (i, j) \in F.$$

目标函数为完成此项工程的时间最短. 因此该问题的数学模型为

$$\begin{aligned} \min \quad & S_{19} \\ \text{s.t.} \quad & S_i + T_i \leq S_j, \quad \forall (i, j) \in F, \\ & S_i \geq 0, \quad \forall i \in G. \end{aligned} \quad (8.13)$$

从先决任务图中可以看出, 完成任务至少需要时间

$$S_{19} = T_1 + T_2 + T_3 + T_5 + T_6 + T_9 + T_{12} + T_{17} + T_{18} = 64,$$

同时可以验证, 从任务 1 到任务 19 的其他路径时间均少于 64 周. 因此该问题的最优解即为 64 周.

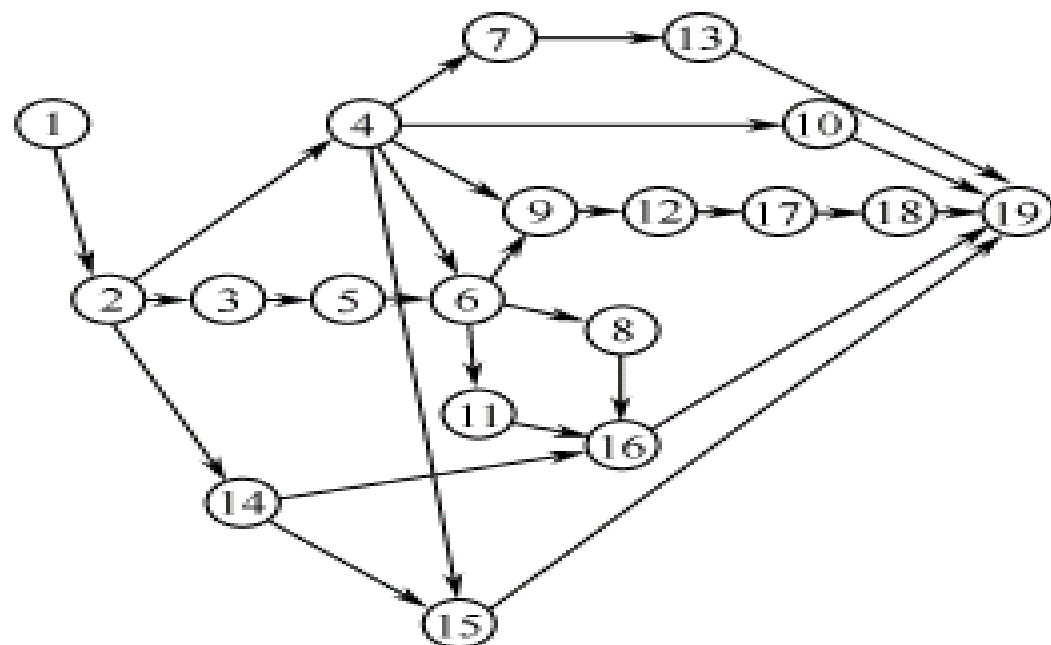
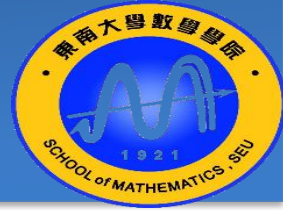


图 8.5 任务先决图



对于问题(2),定义变量 r_i 为任务 i 的缩短时间, e_i 为任务 i 每周额外支出, R_i 表示其最大可缩短时间. 目标函数为最大利润. 任务 j 的开始时间必须不小于任务 i 的结束时间减去其缩短时间. 即

$$S_i + T - r_i \leq S_j, \quad \forall (i, j) \in F.$$

与问题(1)相比,工程缩短时间为 $64 - S_{19}$,

对应的优化问题为

$$\begin{aligned} \max \quad & C = 30(64 - S_{19}) - \sum_{i=1}^{19} e_i r_i \\ \text{s.t.} \quad & S_i + T_i - r_i \leq S_j, \quad \forall (i, j) \in F, \\ & S_i \geq 0, \quad \forall i \in G, \\ & r_i \leq R_i. \end{aligned} \tag{8.14}$$



LINGO 求解可得全局最优解 $C = 87$ (千元) (最优方案不唯一), 如同时考虑工程总工期缩短情况, 经计算最多可以缩减到 54 周.